IMPLEMENTATION ROADMAP / 20 OCTOBER 2026

Avatar kinematics From paired capture to robot learning

First milestone: a reproducible data factory, a working policy, and a measured answer to whether paired views help. Preserve the longer-term human-avatar vision through separate research phases.

Based on the feasibility study for PraxisOG/avatar-world-model at commit 3582489. This is a proposed roadmap: no implementation has been completed or scheduled by creating this document. Version labels below identify planned milestones.

Release	Concrete result	Active effort
v0.1 / Platform proof	Moe captures synchronized views and runs one-GPU training.	8-16 h
v0.2 / Capture workbench	Teleoperation, replay, validation and LeRobot export.	20-40 h
v0.3 / Working baseline	100-300 episodes across three tasks; direct policy evaluated.	16-32 h
v0.4 / Paired-view experiment	Controlled comparison of direct, latent and pixel supervision.	16-32 h
v0.5 / Physical pilot	One SO-101 performs a narrow task; real-only vs mixed data measured.	40-80 h
v0.6 / Reproducible release	Usable package, dataset card, examples, checkpoints and results.	8-16 h

What v0.6 must deliver

A new checkout can generate and inspect an episode, export and reload the data, train a baseline, and reproduce the evaluation protocol. The physical pilot includes documented calibration and trial results. The release remains useful even if paired-view supervision does not outperform the direct policy.

Constraints that keep the project achievable

• One robot arm, one gripper, rigid tabletop objects and two calibrated views. Use an SO-101-compatible embodiment in simulation before physical transfer.

• Start with native ROCm and basic rasterized cameras. Keep MuJoCo as a fallback; avoid dependencies on Nyx, Madrona, photorealism or multi-GPU sharding for the first release.

• Buy hardware only after v0.4. Target a $350-500 complete workbench, with $500 as the cap. A follower plus an existing input device is the cheaper fallback.

Planning totals: 60-120 active hours through v0.4; 108-216 through v0.6. At an assumed 8 hours/week: roughly 2-4 months and 3-6 months respectively. Training runtime, shipping and research setbacks are additional. These are estimates, not measured V620 throughput.

1. Establish the platform and capture contract

v0.1 / Prove the loop on Moe - 8-16 hours

• **Environment:** capture GPU, driver, ROCm, Python and dependency versions. Separate simulation/export from training environments. Record the working combination before adding optional acceleration.

• **Small scene:** load one arm, a table and one object. Use a fixed external camera and a robot-mounted camera, initially at 320x240 or similar. Script a short arm motion.

• **Transition logging:** capture both images from one frozen state; record actual joint state, commanded action and the resulting next state. Include camera transforms, step ID and simulation time.

• **Learning smoke check:** load a tiny recorded sample into a small policy and complete forward, backward and optimizer steps on one V620. Measure a warmed-up 500-1,000-step run if the basic check passes.

Done when: a short episode loads correctly; both camera streams refer to identical simulation steps; training loss is finite; the benchmark records frames/sec, steps/sec, peak VRAM and elapsed time. Real-time rendering is not required for offline scripted collection, but interactive teleoperation must feel usable.

If blocked: within the 8-16-hour budget, simplify the renderer or switch the capture scene to MuJoCo. A CPU-only pass verifies parts of the code, but does not satisfy the V620 training gate. Resolve any remaining GPU blocker before continuing.

v0.2 / Build the capture workbench - 20-40 hours

• **Control:** add keyboard/gamepad or existing-controller teleoperation, reset, episode start/stop and a clear recording indicator. Use scripted motion to exercise the recorder automatically.

• **Durable episodes:** save raw observations/actions, environment seed, robot and object state, asset versions, camera calibration, success/failure and intervention labels. Keep commands distinct from measured response.

• **Replay and inspection:** provide a synchronized viewer with both images, action/state plots and event markers. Support deterministic-within-tolerance state replay; do not require cross-machine pixel identity.

• **Export:** pin a LeRobot format and reader version; export data and prove reload/training compatibility. Keep raw timing and calibration in a companion manifest.

Done when: at least 10 episodes survive capture, save, reload, replay, export and training-load checks without manual file repair. Validation rejects missing frames, non-monotonic timestamps, unknown units, wrong joint ordering and inconsistent episode boundaries.

Non-negotiable schema

Each transition links observation/state at t, action applied over t to t+1, and state at t+1. Record control interval, action units, control mode, joint ordering, gripper meaning and camera intrinsics/extrinsics. Simulation-only object poses and contact labels must be marked as privileged data and excluded from ordinary policy inputs.

2. Establish a baseline, then test the idea

v0.3 / Dataset v0 and direct policy - 16-32 hours

• Create three bounded tasks: reach a target, push an object into a zone, and pick/place into a generous bin. Define success numerically before collection; start with the easiest task.

• Collect 100-300 distinct episodes total, approximately 20-40 seconds each. Vary layouts, object starts, textures and camera placements within declared ranges. Track collection coverage rather than merely frame count.

• Label failures and recoveries. Train initial imitation policies on successful demonstrations and verified recovery segments; do not blindly teach the policy every failed operator action.

• Split by underlying trajectory before rendering additional views. Freeze training, validation and test IDs; keep held-out layouts/camera poses for evaluation.

• Train a direct policy using the external view plus robot proprioception. First overfit a few episodes to catch implementation errors, then train and evaluate on the frozen split.

Done when: the direct baseline reaches the initial engineering target of at least 80% success over 50 held-out trials on the simplest task. Report each task separately, plus failures and action latency. Harder tasks need not reach 80% before the first comparison, but a weak basic-task baseline must be fixed first.

v0.4 / Paired-view ablations - 16-32 hours

Variant	Training change	Deployment observations
A: direct baseline	External image + state predict actions.	External image + state
B: latent alignment	Add a training-only loss aligning information from paired views.	External image + state
C: pixel auxiliary head	Add a training-only head predicting the second camera image.	External image + state
Oracle reference	Train using the true robot-mounted view.	True second view + state; report separately

Reuse the data and splits. Track model size, training time and GPU usage; do not give B/C privileged runtime observations. Evaluate A/B/C with three training seeds and 50 held-out trials per task per seed. Use a preselected viewpoint/occlusion condition if the easiest task has little remaining headroom.

Decision: a repeatable gain of about 10 percentage points in the selected condition, without basic-task regression, is a useful continuation target. Report counts and uncertainty; a small noisy difference is inconclusive. If B/C fail, retain the direct policy and pipeline. If they help, investigate why before scaling collection.

Keep the expensive full exo-to-generated-ego-to-action chain optional. A small pilot can follow a positive result; larger pretrained video-model adaptation has its own compute budget and is not included in the 16-32-hour estimate.

3. Real-arm pilot and reproducible release

v0.5 / Real-arm pilot - 40-80 hours

Purchase gate: v0.4 has produced a trustworthy baseline and a documented paired-view result, including an honest negative result. The pipeline has value without a positive translation result; hardware work must test a specific remaining question.

• **Assemble and characterize:** one SO-101 follower, optional leader, simple RGB cameras and printed mounts. Calibrate joints and camera geometry. Define a bounded workspace, conservative motion limits and an operator stop.

• **Measure the mismatch:** log commanded versus measured position, control rate, camera timestamp skew, execution delay and visible tracking error. Fix loose mounts, joint offsets and timing before changing the model.

• **Real-only baseline:** collect an initial 30-50 good demonstrations of one rigid-object task, such as block-to-bin placement. Expand only to address observed failure modes. Establish a policy trained on real data first.

• **Transfer comparison:** compare real-only training with real-plus-simulation at the same real-data budget. Keep task/layout tests separate from collection. Report any data or calibration added after a failed attempt.

Done when: the narrow real task achieves at least 20 successes in 30 held-out trials, with reset rules, interventions and failures recorded. This is a pilot threshold, not proof of deployment reliability. If differences between training methods are uncertain, collect more evaluation trials before claiming a benefit.

If it fails: diagnose real-only performance first. A bad calibration or poor real demonstration set cannot establish that synthetic data is useless. Conversely, a working real-only policy plus a worse mixed-data policy is evidence to reduce or revise the synthetic contribution.

v0.6 / Reproducible release - 8-16 hours

• Package setup instructions, pinned environments, a minimal example and scripts/configs for capture, validation, export, training and evaluation. Keep large data and checkpoints outside ordinary Git history.

• Prepare a data card with task coverage, provenance, asset/data rights, camera specs, action definitions, known failures and limitations. Keep restricted external data separate from your own releasable material.

• Include frozen split manifests, baseline checkpoints, measured compute/cost logs, success counts and paired-view comparison results. Include a small sample dataset that works without downloading everything.

• Run the documented path from a fresh environment. Prepare a release candidate; publication to GitHub or a dataset host is a separate explicit release action.

Done when: a fresh checkout and the small sample can reproduce the data workflow, and the full experiment has enough configuration and provenance to rerun. The conclusion must state what paired views helped, where they did not, and what remains untested.

4. Human-avatar and world-model research

Research phases after the engineering foundation

Phase	Smallest meaningful experiment	Continue only when
R1 / Human-avatar capture	Add a human arm/hand rig and calibrated head camera. Capture independent motion and use synchronized ego/exo human data for held-out evaluation.	Pose/view metrics are defined against real ground truth; object/contact reconstruction assumptions are explicit.
R2 / Video to robot skill	Infer task-relevant wrist/object motion and grasp events; retarget one simple demonstration with reachability and collision filtering. Compare direct features with generated ego views.	The transfer improves a robot task or reduces real demonstration needs; attractive video alone is insufficient.
R3 / Learned dynamics	Predict short-horizon object/state or latent transitions conditioned on chosen actions. Compare against persistence and simulator/real transitions.	Held-out actions and disturbances are predicted accurately enough to improve planning.
R4 / Recovery and expansion	Use actual simulation for recovery data first. Test learned-model planning only after R3. Add a second arm or richer tasks one at a time.	Measured recovery gains justify extra hardware, data and modeling complexity.

These phases retain the original avatar/world-model ambition. They have no reliable completion dates yet. Their budgets should be set after v0.6 reveals actual throughput, data quality and transfer performance. Robot-wrist view success does not establish human head-view transfer.

Keep the evidence for each claim separate

The robot experiment measures whether synthetic paired views help control the same embodiment. The human-avatar experiment measures whether pose and view estimates match synchronized ground truth. Retargeting measures whether those estimates produce feasible robot actions. Learned dynamics measures whether chosen actions produce correctly predicted outcomes. Success in one is a reason to try the next experiment, not proof that the next is solved.

What should remain reusable

The timestamped episode schema, camera calibration, split manifests, replay viewer and evaluation tools should survive changes to the avatar, simulator or model. Add embodiment-specific fields and adapters instead of replacing the data contract. Human skeleton tracks should retain their original coordinate conventions and provenance; robot control labels must identify the retargeting method and filtering decisions.

When to expand the hardware

Add a second arm, depth sensing or tactile sensing only when a recorded failure shows why the current setup cannot answer the next question. Whole-body balance, legged locomotion, dexterous hands and household-scale autonomy each introduce a new control and data problem. They need their own feasibility gate after the workbench loop is established.

5. First sprint and implementation handoff

First 8-16-hour sprint: implement v0.1 only

Task	Reviewable output
1. Freeze the minimal experiment	One robot, one scene, two cameras; document action/state/time conventions and what counts as a pass.
2. Set up one V620	Record environment versions; verify simulation and graphics devices independently; complete a tensor backward pass.
3. Capture a short episode	A scripted trajectory with two image streams, joint states, commands, next states and calibration.
4. Validate and train	Alignment check, loader check and finite policy optimizer steps. A warm training timing run if the basic path passes.
5. Decide the next build	A concise pass/fail report, measured throughput and peak VRAM, remaining blockers, and either v0.2 scope or the fallback decision.

Ownership during implementation

Codex can implement adapters, validation, exporters, evaluation and documentation. Tests on Moe, calibration, physical data collection and arm trials require access to your equipment or your execution of the scripts. Each release should end with a short result summary and the next measured bottleneck.

Suggested repository work items

Use separate reviewable changes for environment/scene setup, episode schema and recorder, validator/replay, LeRobot export, task definitions/splits, direct policy, paired-view losses, real-arm adapter and release documentation. Propose design corrections through an issue or PR against DESIGN.md, following AGENTS.md. No issues or repository changes were created by this roadmap.

Source: Avatar Kinematics Feasibility Study, 20 October 2026, nine pages, based on repository commit 3582489. Software findings and hardware allowances are carried forward from that study; recheck the chosen versions and delivered prices when implementing or buying. Phase splits, work items and acceptance gates here are planning judgments.